covtl-pipeline

User Manual

Frédéric Berthommier

fberthommier@users.noreply.github.com

Version 1.1.0 — September 2026

Licensed under GPL-3.0-or-later

Abstract

covtl-pipeline is a Python package and command-line tool (`vtl-synth`) that converts raw English text — or direct SAMPA phonetic input — into an articulatory speech signal and an animated video of the vocal-tract sagittal cross-section. Its synthesis engine implements the **Berthommier COVTL articulatory model**: every phoneme carries a target defined by *polar coordinates* (ρ, θ) in a compact two-dimensional phonetic plane, and coarticulation emerges from the *superposition* of a continuous vocalic trajectory with transient consonantal gestures projected onto disjoint sets of VocalTractLab parameters. A tag-based extension engine drives voicing, nasality, laterality, glottal effort and F0, and a syllabic trajectory engine (*syltraj*) shapes the consonant/vowel arcs, inter-word pauses and phrase-final holds. The 400 Hz tract sequence is synthesized *directly* by this model — there is no intermediate gestural-score stage — and audio is rendered by **VocalTractLab 2.4** (Peter Birkholz, TU Dresden) through the `vocaltractlab-cython` wheel with the active registry speaker (default: JD3). This manual documents the model in detail (polar targets, projection coefficients, coarticulation rules), the command-line interface, the Python API, the file formats, and the underlying mechanisms and constants. Version 1.1.0 ships the package as an integrated pack: a five-speaker registry with a one-command reversible selector (`install_speaker.py`), a six-language pack (`setup_lang.py`), an expressive-prosody option (`--expressive`) and a dual-panel polar video (`--polar`); each add-on has its own companion manual in `docs/` (§1.1).

At a glance

- **Pipeline:** text → SAMPA → polar targets (ρ, θ) → 100 Hz model trajectory → `.tract` (400 Hz, 19 + 11 params) → `.wav` (44.1 kHz) → `.mp4`
- **CLI:** `vtl-synth run "this is easy for us" -o ./out`
- **Python API:** `from vtl_synth import Pipeline`
- **Model:** Berthommier COVTL — polar phonetic targets, selector-based coarticulation, S-curve arcs
- **Synthesizer:** VocalTractLab 2.4 via `vocaltractlab-cython`
- **Speakers:** five-speaker registry (jd3, s1, s2, m01, w02) with a one-command reversible selector
- **Languages:** six-language pack en/fr/es/de/it/pt with calibrated vowel targets (default: en)
- **Options:** expressive prosody (`--expressive`) and dual-panel polar video (`--polar`)
- **License:** GPL-3.0-or-later (inherited from upstream VTL components)

Contents

1 Introduction **4**
1.1 Documentation map . . . 5

2 Quick start **5**
2.1 Prerequisites . . . 5
2.2 Installation . . . 5
2.3 First run . . . 6
2.4 Inspecting the output . . . 6

3 The COVTL model I: phonetic targets in polar coordinates **7**
3.1 The (ρ, θ) phonetic plane . . . 7
3.2 Vowel targets . . . 7
3.3 Consonant targets and selectors . . . 8
3.4 Context-conditioned targets . . . 10
3.5 Projection onto the VocalTractLab parameters . . . 10

4 The COVTL model II: coarticulation **11**
4.1 Superposition of the vocalic and consonantal branches . . . 11
4.2 Lightened selectors: the tongue during labials . . . 12
4.3 The transition arc: an asymmetric S-curve in the plane . . . 12
4.4 Clusters: per-parameter scoping . . . 13
4.5 Vowel-to-vowel coarticulation and the schwa waypoint . . . 13
4.6 Coarticulation across word boundaries . . . 13
4.7 Nasal carry-over . . . 14

5 The syllabic trajectory engine (syltraj) **14**
5.1 The orthogonal branch . . . 14
5.2 The legacy engine . . . 14

6 The source model: envelopes and the glottal source **15**
6.1 The three-level tag system . . . 15
6.2 The amplitude envelope $E(t)$ and the source timetable . . . 15
6.3 Voicing, VOT and nasality . . . 15
6.4 Glottal source, pressure and effort . . . 16
6.5 F0: declination and micro-prosody . . . 16

7 The multi-speaker registry and the language pack **17**
7.1 The speaker registry . . . 17
7.2 The language pack . . . 17

8 Architecture and data flow **18**
8.1 Data flow . . . 18

9 CLI reference **19**
9.1 run — all-in-one synthesis . . . 19
9.2 text-to-wav — audio-only synthesis . . . 20
9.3 tract-to-mp4 — re-render a video . . . 20
9.4 polar-build / polar-video — the polar figure . . . 20
9.5 inspect — tract inspector . . . 20
9.6 plot-tract — graphical inspector . . . 20

10 Python API reference **21**
10.1 Pipeline — high-level orchestrator . . . 21
10.2 Video re-rendering . . . 21
10.3 Engine and VTL APIs . . . 21

11 Input notation and grapheme-to-phoneme conversion **22**
11.1 Engine SAMPA inventory . . . 22
11.2 English text rules . . . 22

12 File formats reference **23**
12.1 `.tract` — tract sequence at 400 Hz . . . 23
12.2 `.wav`, `.mp4`, `.txt` . . . 23

13 Worked example: “this is easy for us” **24**
13.1 g2p output . . . 24
13.2 Tract sequence . . . 24
13.3 Audio and video . . . 24

14 Differences with the vtl-pipeline reference repository **25**

15 Limitations and known issues **25**

16 Developer guide **26**
16.1 Package layout . . . 26
16.2 Running the tests . . . 26
16.3 Extending the model . . . 26

17 License, acknowledgments and references **27**

1 Introduction

covtl-pipeline is an articulatory text-to-speech and video-synthesis pipeline. Given raw English text, it produces:

- a phonemic transcription in the engine SAMPA notation,
- a tract sequence (`.tract`) giving the 19 VocalTractLab tract parameters and 11 glottis parameters at 400 Hz,
- an audio waveform (`.wav`) at 44 100 Hz, synthesized by VocalTractLab,
- an animated MP4 showing the sagittal cross-section of the vocal tract synchronized with the waveform.

The synthesis engine is the **Berthommier COVTL articulatory model**, a compact mathematical model of speech production in which phonetic knowledge is encoded geometrically:

- every phoneme — vowel or consonant — is a point $z = \rho\, e^{i\theta}$ in a *polar phonetic plane*; the radius ρ is an aperture/closure magnitude and the angle θ is the direction of the articulatory gesture;
- the plane is projected linearly onto the 15 controlled parameters of a Maeda-type vocal-tract model, so that a single (ρ, θ) target simultaneously coordinates jaw, lips, hyoid, tongue body, tongue tip, tongue base and tongue sides;
- coarticulation is not a post-hoc smoothing: it is the structural *superposition* of a continuous *vocalic branch* and transient *consonantal branch*, each projected onto its own subset of parameters (its *selector*). During a labial stop the tongue keeps following the vowel; during a lingual consonant the jaw and lips keep following it;
- source characteristics (voicing, nasality, laterality, glottal effort, F0) are driven by a tag-based extension engine whose events are *indexed by the articulatory trajectory itself* — the source inherits the coarticulation of the tract.

The model is coupled to **VocalTractLab 2.4** (VTL), the physiological articulatory synthesizer developed by Peter Birkholz and collaborators at TU Dresden. VTL converts each 400 Hz vocal-tract state into cross-sectional areas and runs its aerodynamic-acoustic simulation to produce the 44.1 kHz waveform. The package depends on the `vocaltractlab-cython` wheel (Paul Krug's Cython binding of VTL), which also provides the SVG export of vocal-tract shapes used by the video pipeline. The repository and packaging layout follow the vtl-pipeline reference repository; the synthesis path itself is entirely different (see §14).

Key features

- End-to-end pipeline from raw text to MP4 in a single command, or direct SAMPA input with `--phonetic`.
- Polar-coordinate phonetic targets with documented, calibrated coefficient tables (§3).
- Selector-based coarticulation with lightened selectors, contextual consonant targets, cluster scoping and vowel-to-vowel background arcs (§4).
- Tag-based source model: voicing, VOT by place of articulation, nasality with carry-over, glottal effort and F0 declination (§6).

- Syllabic trajectory engine (`syl`) with pause handling and phrase-final holds; a deprecated pre-`syltraj` alignment engine (`legacy`) remains selectable (§5).
- Inspectors for the tract sequence: `inspect` (tables, ranges) and the graphical `plot-tract`.
- Per-speaker non-regression baselines (jd3, s1, s2, m01, w02) checked by pytest; the repository ships a lightened quick suite (§16).
- Four integrated add-ons — multi-speaker registry, language pack, expressive prosody, polar video — each documented in its own companion manual (§1.1).

1.1 Documentation map

The `docs/` folder carries five manuals, written to be read independently:

`manual.pdf` this reference manual: the COVTL model, the CLI and Python API, the file formats.

`speakers.pdf` the multi-speaker registry: the five speakers and their provenance, the selector `install_speaker.py`, and the two adaptation chains (DVTD construction; official-speaker patching).

`language_pack.pdf` the language pack: six languages, installation, switching, calibration and verification through `setup_lang.py`.

`expressive_prosody.pdf` the expressive-prosody option: syntactic breathing and sculpted F0 contour.

`polar_visualization.pdf` the dynamic polar video: two-branch display of the articulation, `.polar` format, dual-panel MP4.

2 Quick start

2.1 Prerequisites

- Python 3.9 or newer.
- No system `ffmpeg` is required: the `imageio-ffmpeg` wheel bundles a private ffmpeg binary for MP4 encoding.
- About 200 MB of disk for the Python packages.

2.2 Installation

```
git clone https://github.com/fberthommier/covtl-pipeline.git
cd covtl-pipeline
pip install .            # or: pip install -e .   (development)

# Optional: PNG figures of .tract files (plot-tract)
pip install ".[plot]"    # pulls matplotlib

# Verify
vtl-synth --help
```

The install pulls `numpy`, `scipy`, `vocaltractlab-cython`, `Pillow`, `imageio-ffmpeg` and `cmudict` automatically. The console script `vtl-synth` lands in the Python *Scripts* folder; with `pip install --user` on Windows this is `%APPDATA%\Python\Python39\Scripts` — add it to `PATH`, call it by full path, or use the provided `launch.bat`. Under Python 3.9, if pip tries to build `vocaltractlab-cython` from source, install `vocaltractlab-cython==0.0.13` instead (full synthesis and video path available; four optional analysis helpers raise a clear error). The polar video (`--polar`) and `plot-tract` additionally require `matplotlib` (`pip install ".[plot]"`).

2.3 First run

The shipped tree is delivered in the reference state (speaker `jd3`, language `en`). The `ACTIVE_SPEAKER` marker is not part of the package: create it once before the first synthesis with `python install_speaker.py list` (§7).

```
vtl-synth run "this is easy for us" -o ./out
```

This produces the following artifacts in `./out/` (real output, default configuration):

```
this_is_easy_for_us.tract   268,286 bytes   952 frames x (19 tract + 11 glottis)
    @ 400 Hz
this_is_easy_for_us.wav     209,484 bytes   PCM 16-bit, mono, 44100 Hz, 2.37 s
this_is_easy_for_us.mp4     239,001 bytes   H.264 960x840 @ 25 fps + AAC, 2.40 s
this_is_easy_for_us.txt          20 bytes   the SAMPA transcript:
    Dis.iz.i.zi.fOR.@s
```

The video shows the sagittal cross-section of the vocal tract; the audio is normalized to 90 % of full scale by the pipeline.

2.4 Inspecting the output

```
# Summary + per-parameter min/max/mean of a .tract
vtl-synth inspect ./out/this_is_easy_for_us.tract --states 2 --ranges

# Graphical counterpart: 400 Hz trajectory of each parameter
vtl-synth plot-tract ./out/this_is_easy_for_us.tract --which all -o
    ./out/tract.png
vtl-synth plot-tract ./out/this_is_easy_for_us.tract --params JX JA TTX TTY

# Re-render a video from an existing tract + wav (no re-synthesis)
vtl-synth tract-to-mp4 ./out/this_is_easy_for_us.tract
    ./out/this_is_easy_for_us.wav \
    -o ./out/this_is_easy_for_us_hd.mp4
```

3 The COVTL model I: phonetic targets in polar coordinates

This section and the next describe the core of the synthesis engine: how phonemes are represented, how the plane is mapped onto vocal-tract parameters, and how coarticulation is obtained. The reference publications are listed in §17; the implementation lives in `vtl_synth/core/polar.py`, `projection.py`, `constants.py` and `syltraj.py`.

The global architecture of the model is a sum of three contributions:

$$\mathrm{VTL}(t) \;=\; B(t) \;+\; E_{\mathrm{source}}(t) \;+\; E_{\mathrm{ortho}}(t), \tag{1}$$

where $B(t)$ is the *Berthommier polar core*: the 15 COVTL tract parameters trajectory built from polar targets; $E_{\mathrm{source}}(t)$ groups the extension parameters produced by the tag/envelope engine (velum VS/VO and the 11 glottis parameters); and $E_{\mathrm{ortho}}(t)$ is the orthogonal branch (TS3 tongue-side shape and the speaker-derived VS), read from the JD3 speaker file (§5.1).

3.1 The (ρ, θ) phonetic plane

Every phoneme is a point

$$z \;=\; \rho\, e^{\,i\theta}, \qquad \rho \geq 0,\ \theta \in [0, 2\pi), \tag{2}$$

in a complex *phonetic plane*:

- ρ is a dimensionless **aperture/closure magnitude**. $\rho = 0$ is the origin (minimum displacement); $\rho \leq 1$ covers the vowel space; $\rho > 1$ — beyond the unit circle — is the *constriction/occlusion region* used by consonants. The per-parameter amplitude c_0 (Table 3) carries the physical (centimeter) scaling.
- θ is the **direction of the articulatory gesture**. The convention is fixed by the three corner vowels of the Maeda reference vowel triangle: /a/ $= \pi$ (180°, low-open-central), /i/ $= 5\pi/3$ (300°, palatal-front), /u/ $= \pi/3$ (60°, velar-back-round).

Figure 1 shows the plane drawn to scale from the target tables. Two properties are worth noting:

- **Vowels are anchors.** A vowel target becomes a stationary plateau (a *hold*) of the trajectory whenever it is not bracketed by an immediate transition; $\rho \leq 1$ keeps the tract open.
- **Consonants are arc endpoints.** A consonant is never interpolated from the previous consonant — each keeps its own target — and ρ typically ranges from 1.0 to 1.45, i.e. the gesture goes *through* the unit circle to reach a constriction or a complete closure (occlusion areas of $0.000\,\mathrm{cm}^2$ are reached and verified in the test suite).

3.2 Vowel targets

The twelve vowel targets of the engine (`VOWEL_TARGETS` in `constants.py`) are given in Table 1. The corner vowels /a/, /i/, /u/ are the strict Maeda anchors; the other vowels were fitted by least squares under the constraint $\rho \leq 1$. Two documented compensations: the center coefficient c_1(TCY) had to be raised by +0.25 to obtain the velar occlusion of /g/ (Table 3), and the radii of /i/ and /u/ were reduced (0.80 → 0.60 and 0.80 → 0.55) to compensate, preserving F2 to within a few tens of Hz. The schwa @ is placed at the centroid of the calibrated a–i–u triangle, $\bar{\rho} = (1.000 + 0.600 + 0.550)/3 = 0.7167$, $\theta = \pi$; it serves as the mute vowel of isolated consonants and as a waypoint against vowel-to-vowel closure (§4.5).

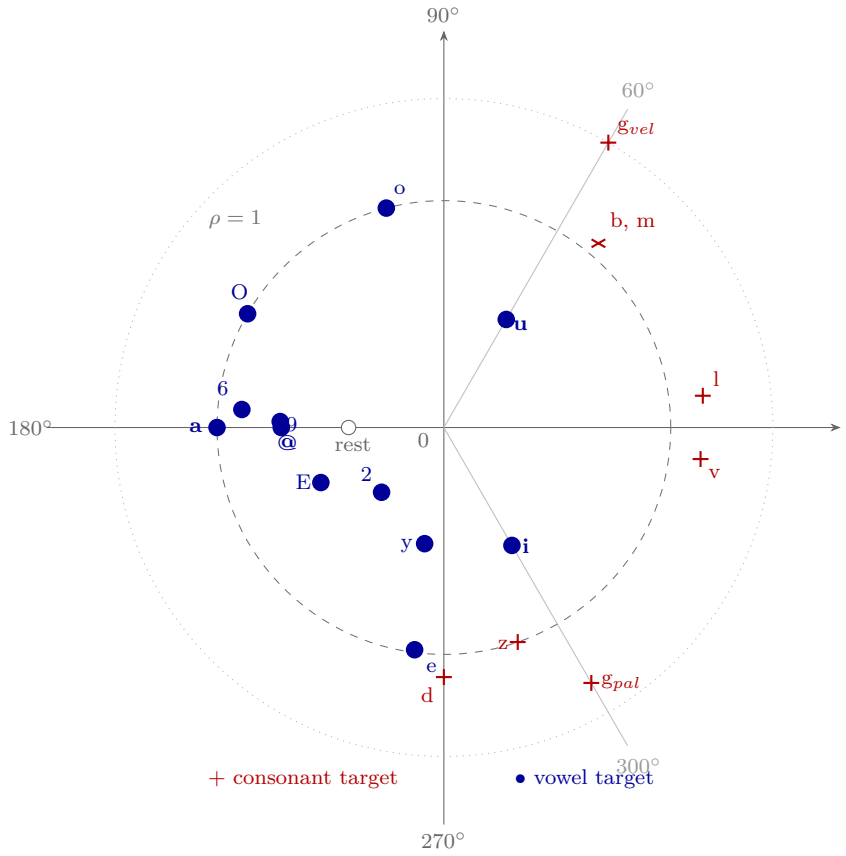


Figure 1: The (ρ, θ) phonetic plane of the COVTL model, drawn to scale from the target tables of `constants.py`. Filled dots: the 12 vowel targets (Table 1); crosses: landmark consonant targets (Table 2). The dashed unit circle is the frontier of the vowel space; consonant closures live beyond it ($\rho > 1$). The schwa @ sits at the centroid of the calibrated $a{-}i{-}u$ triangle. /m/ coincides with /b/ — same target and same selector; only the velum extension (VO, outside the COVTL selector) differs.

3.3 Consonant targets and selectors

A consonant target is a triple $(\rho, \theta, \mathcal{S})$: a polar point *plus a selector* $\mathcal{S}$, the list of COVTL parameters that the consonant is allowed to command. Parameters outside $\mathcal{S}$ are *not* touched by the consonant — they keep following the vocalic branch. This is the structural basis of coarticulation (§4).

Two conventions are essential:

- **Voicing is not in the tract.** Only the canonical voiced form of each obstruent is tabulated; the voiceless keys `p, t, k, f, s, S, T, tS` are normalized to their voiced partners (`p`→`b`, `t`→`d`, `k`→`g`, `f`→`v`, `s`→`z`, `S`→`Z`, `T`→`D`, `tS`→`dZ`) before target lookup, because a voicing pair shares the same place of articulation — hence the same (ρ, θ) and the same selector. The voicing distinction is carried entirely by the glottal source (§6).

- **Glides are consonantal nodes anchored on the reference vowel tree.** `j` is anchored at the reference coordinates of `i` $(0.80, 300°)$, `w` at those of `u` $(0.60, 60°)$ and `ɥ` at those of `y` $(0.519, 260.6°)$; they are treated as C nodes (no plateau), with selectors restricted accordingly. These anchor radii are the reference-model values, deliberately kept independent of the calibrated vowel plateaus of Table 1 (where $\rho_i = 0.60$ and $\rho_u = 0.55$): a glide is a consonantal target, not a vowel plateau.

key	$\boldsymbol{\rho}$	$\boldsymbol{\theta}$ (rad)	$\boldsymbol{\theta}$ (deg)	**note**
`a`	1.000	π	180.0	strict Maeda anchor
`i`	0.600	$5\pi/3$	300.0	anchor, ρ compensated
`u`	0.550	$\pi/3$	60.0	anchor, ρ compensated
`e`	0.988	4.581	262.5	
`E`	0.594	3.563	204.1	
`o`	1.000	1.827	104.7	clamped at $\rho = 1$
`O`	1.000	2.616	149.9	clamped at $\rho = 1$
`6`	0.894	3.053	174.9	œ
`9`	0.722	3.105	177.9	œ̃ (nasalized via VO)
`@`	0.7167	π	180.0	centroid of the *a*–*i*–*u* triangle
`y`	0.519	4.548	260.6	
`2`	0.396	3.946	226.1	ø

Table 1: Vowel targets `VOWEL_TARGETS` (`constants.py`).

key	(ρ, θ)	**selector**	**note**
`b`	$(1.06, 50°)$	JA, LD, LP	labial stop; lightened selector (§4.2)
`d`	$(1.10, 270°)$	JA, TCX, TCY, TBX, TBY, TTX, TTY	dental/alveolar stop
`g`	$(1.45, 60°)$	JA, TCX, TCY, TBX, TBY	velar stop; conditional → `g_vel`/`g_pal`
`g_vel`	$(1.45, 60°)$	JA, TCX, TCY, TBX, TBY	velar occlusion 0.000 cm^2
`g_pal`	$(1.30, 300°)$	JA, TCY, TBX, TBY	palatal; TCX removed from the selector
`v`	$(1.14, 353°)$	JA, JX, LP, LD	labiodental; lightened selector
`z`	$(1.00, 289°)$	JA, TCX, TCY, TBX, TBY	alveolar fricative
`z_front`	$(0.90, 289°)$	JA, TCX, TCY, TBX, TBY	contextual variant before `i`, `u`
`Z`	$(1.02, 351°)$	JA, TCX, TBX, TBY, LP	postalveolar; TCY removed (coarticulation)
`D`	$(1.15, 271°)$	JA, TCX, TCY, TBX, TBY	dental fricative
`R`	$(1.28, 234°)$	JA, TCX, TCY, TBX, TBY, TS2	uvular
`dZ`	$(1.15, 7°)$	JA, TCX, TCY, TBX, TBY, LP	affricate (canonical voiced)
`l`	$(1.15, 7°)$	JA, TCX, TCY, TBX, TBY	lateral (laterality via TS3)
`L`	$(1.25, 7°)$	JA, TCX, TCY, TBX, TBY	dark lateral
`C`	$(1.15, 353°)$	JA, TCX, TCY	palatal fricative (ç)
`X`	$(1.28, 234°)$	JA, TCX, TCY, TBX, TBY, TS2	uvular fricative
`m`	$(1.06, 50°)$	JA, LD, LP	labial nasal; velum via VO extension
`n`	$(1.10, 7°)$	JA, TCX, TCY, TBX, TBY, TTX, TTY	alveolar nasal

key	(ρ, θ)	selector	note
`N`	$(1.20, 60°)$	(empty)	derived velar place (no closure target)
`J`	$(1.20, 300°)$	(empty)	derived dorsal place; the English velar `NG` maps here (documented compromise, Table 5)
`w`	$(0.60, 60°)$	JA, LP, LD, TCX, TCY	glide node on the reference vowel tree
`j`	$(0.80, 300°)$	JA, TCX, TCY	glide node on the reference vowel tree
`ɥ`	$(0.519, 260.6°)$	JA, TCX, TCY	glide node on the reference vowel tree
`h`	$(0.50, 180°)$	(empty)	near-neutral; source-driven

Table 2: Consonant targets `CONSONANT_TARGETS` (`constants.py`), canonical voiced forms. The selector is the set of COVTL parameters commanded by the consonant; all other parameters follow the vocalic branch. Voiceless keys resolve to the voiced partner before lookup.

3.4 Context-conditioned targets

The tabulated target is only the *canonical* form; the engine adjusts it from context, in two ways:

Conditional places. $/g, k/$ choose between `g_vel` and `g_pal` from the vowel context: a following vowel with $\theta > \pi$ (front) triggers the palatal target, otherwise the velar one. Similarly `z_front` replaces `z` before the high vowels `i` and `u` (ρ 1.00 $\rightarrow$ 0.90 to keep the alveolar groove open in the high-tongue context).

Cluster rules. `adjust_consonant_theta` applies the reference place rules inside consonant clusters: `d/t` retracts to the cluster angle $\theta_{CC,d} = -2.5\pi/6$ with $\rho = 1.2$, `g/k` uses $\theta_{CC,g} = -\pi/12$ with $\rho = 1.1$, and the liquids/nasals `l, n` are realized stop-like ($\rho_{\text{stop}} = 1.2$, $\theta = 270°$). Outside a cluster, `d/t` use the dental angle $3\pi/2 = 270°$ (v1.2.0: the former $23\pi/16$ override left the apical closure open on every speaker, the reference jd3 included), and `g/k` back to $\theta_{\text{velar,back}} = 23\pi/12$ after a back vowel.

3.5 Projection onto the VocalTractLab parameters

Level 2 of the model — the *coordination* — maps the phonetic plane onto the 15 COVTL-controlled parameters of a Maeda-type vocal-tract model (the remaining 4 of the 19 VTL tract parameters are extensions: velum VS/VO and the auto-driven tongue root TRX/TRY). For a static target (ρ, θ), each parameter p receives

$$P_p(\rho, \theta) \;=\; c_{1,p} \;+\; \rho\, c_{0,p}\, \cos(c_{2,p} - \theta), \tag{3}$$

or, in complex form with $\Psi_p = c_{0,p}\, e^{\,ic_{2,p}}$ and $z = \rho e^{i\theta}$,

$$P_p \;=\; c_{1,p} \;+\; \mathrm{Re}\big[\Psi_p\, \overline{z}\,\big]. \tag{4}$$

Geometrically: the center c_1 is the parameter value at the origin of the plane; the lever c_0 is the amplitude of the excursion; the phase c_2 is the gesture direction in which parameter p is *most* modified (the maximum of Eq. (3) is reached at $\theta = c_2$). A single θ therefore coordinates all 15

param	c_1	c_0	c_2 (rad)	**role**
HX	0.722233	0.555533	0.000000	hyoid position (x)
HY	−5.155867	1.246761	3.361756	hyoid position (y)
JX	−0.025000	0.050000	2.094395	jaw advance
JA	−3.371067	1.299456	5.354986	jaw aperture
LP	0.382967	0.617041	1.042238	lip protrusion
LD	0.466400	0.583000	4.092457	lip distance
TCX	1.090767	1.528173	5.373564	tongue center (x)
TCY	−0.914133	0.690945	6.031444	tongue center (y)
TTX	4.044400	0.832384	4.466027	tongue tip (x)
TTY	−0.459700	1.084300	0.116283	tongue tip (y)
TBX	2.795833	1.272045	4.907833	tongue base (x)
TBY	0.159033	0.836452	0.076068	tongue base (y)
TS1	0.248667	0.049433	0.545179	tongue shape 1
TS2	0.031067	0.032478	4.484628	tongue shape 2
TS3	0.032933	0.119409	2.897420	tongue shape 3

Table 3: Projection coefficients `CO_VTL` (`constants.py`; tuple order in the code is (c_1, c_0, c_2)). Documented calibration shifts: LD c_1 scaled $\times 0.8$ (avoid negative lip distance at high ρ); TCY $c_1 + 0.25$ (velar occlusion of /*g*/); TTY $c_1 + 0.15$ (apical occlusion of /*d, t*/; Maeda origin −0.6097).

parameters at once — this is how jaw, lips, hyoid and the four tongue degrees of freedom move together from one phonetic direction.

As an example, the jaw aperture during any target (ρ, θ) is $\mathrm{JA} = -3.3711 + 1.2995\,\rho\cos(5.3550 - \theta)$: at $\theta = 5.355$ rad ($\approx 307°$) the jaw is maximally open for a given ρ — which is why the palatal-open direction also opens the jaw. The three calibration shifts of Table 3 are the price of coupling the Maeda plane to the JD3 speaker anatomy; they were validated against closure areas ($0.000\,\mathrm{cm}^2$ occlusions) and formant targets.

4 The COVTL model II: coarticulation

Coarticulation — the fact that successive phonemes overlap and deform each other — is not applied as smoothing on a finished trajectory. It is built into the trajectory generation, at three cooperating levels: *superposition* (who commands which parameter), *arc shape* (how fast the handover between targets happens), and *anchor scoping* (how much of a vowel survives at syllable margins).

4.1 Superposition of the vocalic and consonantal branches

The tract trajectory is the superposition of two complex trajectories evaluated in the phonetic plane:

- the **vocalic branch** $z_v(t)$: a continuous chain of arcs joining vowel anchors (*background*);
- the **consonantal branch** $z_c(t)$: transient excursions from the background towards the consonant targets.

Level 3 of the model composes them parameter by parameter:

$$P_p(t) \;=\; S_p^v\,\mathrm{Re}\big[\Psi_p\,\overline{z_v(t)}\,\big] \;+\; S_p^c\,\mathrm{Re}\big[\Psi_p\,\overline{z_c(t)}\,\big], \qquad S_p^v + S_p^c = 1, \tag{5}$$

where the selector of the current consonant defines the binary masks: $S_p^c = 1$ iff $p \in \mathcal{S}$, else $S_p^v = 1$. During a consonant, the parameters of its selector follow the consonantal branch while *all the others keep following the vocalic background*. When no consonant is active, Eq. (5) reduces to the pure vocalic projection of the background arc.

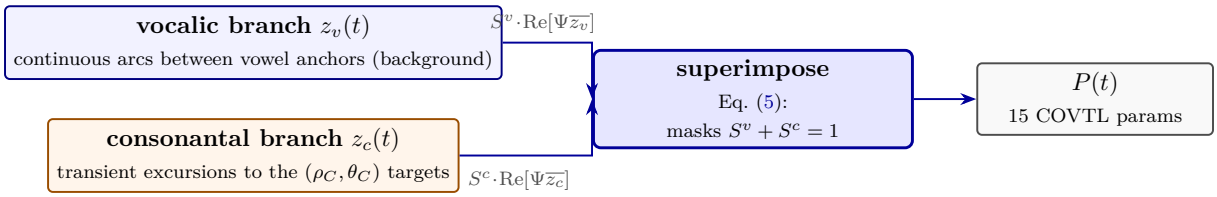


Figure 2: The two-branch superposition of Eq. (5). Each tract parameter belongs to exactly one branch at each instant: the consonant selector decides which.

4.2 Lightened selectors: the tongue during labials

The selectors were deliberately *shortened* to increase coarticulation, with measured justifications recorded in the source:

- **`b` (and `m`)**: TCX/TCY removed — the labial closure only needs the jaw (JA) and the lips (LD, LP). Without tongue parameters in the selector, *the tongue follows the vocalic branch during the labial*; the parasitic velar contact that the raised c_1(TCY) otherwise produced disappears, with the labial closure itself preserved (0.000 cm^2, lingual margins 0.126–1.056 cm^2).
- **`f, v`**: same lightening (TCX and TCY removed); the labiodental constriction is commanded by JA, JX, LP, LD only.
- **`S, Z`**: TCY removed — the postalveolar target projects a tongue center that is too high on the JD3 anatomy; by letting the center follow the vowel, healthy gaps of $0.263 - 1.230\,\mathrm{cm}^2$ are obtained over the whole vowel set while the hushing spectrum is preserved.

The partial removal of a single parameter is not sufficient; the lightening was validated parameter pair by parameter.

4.3 The transition arc: an asymmetric S-curve in the plane

Between a departure point $p_d = (\rho_d, \theta_d)$ and an arrival point $p_a = (\rho_a, \theta_a)$, the engine does not interpolate linearly. It sweeps an angular parameter ϑ from $-\pi$ to 0 over D steps and blends the two projections with the weight

$$r(\vartheta) \;=\; \cos^{p_{\mathrm{exp}}}\!\Big(\frac{\vartheta}{2}\Big), \qquad p_{\mathrm{exp}} = 2, \tag{6}$$

which grows from 0 to 1 as an asymmetric S-curve (slow departure, fast arrival). The arrival term carries an extra phase $-(\nu/K)\vartheta$ that curves the path in the phonetic plane:

$$P_p(k) \;=\; c_{1,p} \;+\; r_k\,\rho_a\,c_{0,p}\cos\!\big(c_{2,p} - \theta_a - \tfrac{\nu}{K}\vartheta_k\big) \;+\; (1 - r_k)\,\rho_d\,c_{0,p}\cos\!\big(c_{2,p} - \theta_d\big), \tag{7}$$

for $k = 1 \ldots D$. This is `arc_B` in `polar.py` (implemented exactly as written; the arrival θ is `np.unwrap`ped first so that the $\pi \to -\pi$ wrap-around never produces a spurious 2π jump). The curvature constants distinguish the branches of the reference model: the consonantal arcs use $K_c = 10$, $\nu = 1$, while the vocalic background uses $K_v = 30$ — the vocalic handovers are much straighter. The active `syl` engine runs with $\nu = -1$ and a quasi-straight $K = 1000$ (§5); this is a notational convention only — $\nu = 1$ is recovered when the cosine is written with the opposite sign, as in Eq. (2) of arXiv:2307.02299. The reference values ($K_c = 10$, $K_v = 30$, $\nu = 1$) belong to the deprecated `legacy` engine and the polar-video display.

4.4 Clusters: per-parameter scoping

For a sequence of m consonants between two vowel anchors, the `syl` engine builds a block of $(m+1)$ sub-arcs of T_{cons} steps each (`build_cluster_pval`). The vocalic background arc between the two anchors is computed first and fills the whole block; then, for each sub-arc k, each parameter's endpoints are set to the consonant target $\mathbf{C}_k$ *iff* the parameter belongs to the selector of $\mathbf{C}_k$ — otherwise the parameter keeps its background value at that frame. The temporal mixing inside each sub-arc uses exactly the reference S-curve of Eq. (6):

$$P[kT+t] \;=\; r_t \cdot \text{arr} \;+\; (1-r_t)\cdot \text{dep}, \qquad t = 1 \ldots T, \tag{8}$$

which guarantees C^0 continuity at every sub-arc boundary (zero jump, verified by test). The active parameter set of the block is the *union* of the selectors of its consonants; a "selector frozen at $\theta = 0$" never drives a parameter in a palatal context.

4.5 Vowel-to-vowel coarticulation and the schwa waypoint

Anchors are reduced at syllable margins. A vowel never reaches its full target ρ_v when it is on the edge of a syllable: the departure anchor (anticipatory) and completion anchor are scaled ($\delta_o = \delta_e = 0.70$ in the reference; SYL_COEFCEN $= 0.5$ for word-end and syllabic-onset anchors in the `syl` engine), and a word-onset anchor is $[0.5\,\rho_v, \theta_v]$. This is why a unstressed vowel between two consonants is *colored* by its neighbors instead of being fully realized.

The background never jumps. Between two anchors the vocalic background is a single continuous arc of $2\,T_{\text{voy}}$ steps; an earlier conditional jump at vowel onsets was removed precisely because it created discontinuities.

Anti-closure routing through the schwa. A direct polar arc between two distant vowels can close the vocal tract mid-transition (the superposition of the two anchor contributions sweeps through the occlusion region — measured on the `i--u` pair: $0.062\,\text{cm}^2$ at mid-arc over 132 swept vowel pairs). The engine hooks a VTL *closure probe* (minimum lingual tube area at the center of the block, on the 19-parameter resting state); when the probe returns less than the threshold $0.18\,\text{cm}^2$, the background arc is routed through the schwa @ as two half-arcs with exact continuity at the junctions.

4.6 Coarticulation across word boundaries

The input notation is itself a coarticulation control:

- `.` (and `-`) is the *intra-phrase syllable separator*: the engine coarticulates across it. Within a phrase, orthographic words are joined with `.` after syllabification by onset maximization, so `this is easy` becomes one connected token `Dis.iz.i.zi` — there is *no pause at word boundaries inside a phrase* (the syllabification rule is the model's answer to "why can *big.bi* be changed to *bi.gbi*": a single intervocalic consonant attaches to the next onset).
- splitting at `.` obeys the phonotactic rule **C.C → split; C.V, V.C, V.V → merge** (the dot is ignored between a consonant and a vowel);
- *space* is a short inter-word pause (200 ms default), `|` is the long phrase pause (320 ms default); a comma produces a space and sentence punctuation produces `|`;
- the *schwa principle* caps long runs: a dot is inserted after the first consonant of any 4+-consonant run and after the first vowel of any 3+-vowel run, and a vowel-less word receives a mute `@`.

Block kind	Content and duration
`plateau`	vowel hold, T_{voy} steps
`background`	vocalic arc between anchors, $2\,T_{\text{voy}}$ steps
`cluster`	m consonants, $(m+1)\,T_{\text{cons}}$ steps (§4.4)
`pause`	short (T_{pause}) or long ($T_{\text{pause,long}}$) inter-word pause
`decay` / `attack`	fall to / rise from the previous anchor around a pending pause, T_{cons} steps
`initial` / `terminal`	phrase bracketing from/to $[0, 0]$

Table 4: Block types emitted by the `syl` engine. Time is counted in gesture steps at the 100 Hz gesture rate (1 step = 10 ms); with the CLI defaults $T_{\text{cons}} = 10$ steps (100 ms), $T_{\text{voy}} = 8$ steps (80 ms, transition arc $2T_{\text{voy}}$), short pause 20 steps (200 ms), long pause 32 steps (320 ms).

4.7 Nasal carry-over

Nasality coarticulates *beyond* the nasal segment: velum-opening (VO) events are extended 20 ms past the end of the nasal (`NASAL_CARRYOVER_MS`), and the whole VO trajectory is smoothed with a 20 ms first-order filter — an anti-click margin over VTL's own 12 ms gesture time constant. Nasalized vowels (`9`, `6~`) receive a partial velum opening of 0.5 (vs 1.0 during nasal consonants).

5 The syllabic trajectory engine (syltraj)

The `syl` engine (`core/syltraj.py`, the default) turns a normalized phoneme string into the 100 Hz polar trajectory $B(t)$. Its contract follows the reference COVTL specification:

1. **Nodes**: every phoneme carries its own polar target; no consonant position is ever interpolated from another consonant; targets are possibly adjusted by the cluster rules (§3.4).
2. **Anchors**: vowels only (held when isolated) plus synthetic *vocalic* anchors: word end (COEFCEN $\cdot\,\rho$ of the last vowel), syllabic onset (weighted ρ), word onset ($[0.5, \theta_v]$), and the terminals $[0, 0]$ that bracket the phrase.
3. **Trajectory**: a walk over consecutive anchor pairs (`build_global_pval`), emitting typed blocks (Table 4).

The block schedule is the single time base shared by the tract and the source: the envelope engine consumes `block_info` directly (§6), a deliberate design that guarantees perfect branch synchronization. A phrase-final vowel followed by nothing holds its plateau (*phrase-final hold*); a pause is *not* a transition to a neutral position — the tract holds the decaying anchor.

5.1 The orthogonal branch

Three of the 19 VTL tract parameters are not commanded by the polar model: **TS3** (tongue-side/lateral shape) and the speaker-derived **VS** (velic). They come from the *orthogonal branch* (`core/orthogonal_params.py`), which reads context-conditioned targets from the `JD3.speaker` file: e.g. fricatives push TS3 to +1.0, laterals to −1.0, closures to 0, with per-vowel defaults looked up with the key pattern `<prefix>(<vowel>)` and a `(a)` fallback. TS3 ramps over 10 ms, VS over 15 ms. The branch is additive on TS3 and priority-based on VS, and can be disabled with `--no-ortho` (the `syl` COVTL core then works alone).

5.2 The legacy engine

The `legacy` engine (`--engine legacy`) is the pre-`syltraj` gesture/anchor chain: it parses the phrase into gesture nodes and anchors (`core/gesture.py`) and walks the anchor pairs with

the block builder `build_global_pval` of `core/trajectory.py` (plateaus at anchors, `arc_B` transitions), with the reference constants $K_v = 30$, $K_c = 10$, $\nu = 1$, $p_{\text{exp}} = 2$ and fixed block timing ($T_{\text{cons}} = T_{\text{voy}} = 160$ ms, pauses 100/200 ms — not parameterizable, unlike the active engine). It is deprecated (a `DeprecationWarning` is emitted) and kept as an alignment reference for non-regression. The former superposition-graph executor of this chain (explicit CV/CVC/CCV graphs in the complex plane) was removed: synthesis follows a single trajectory algorithm per engine, guarded by `tests/test_single_trajectory_path.py`.

6 The source model: envelopes and the glottal source

The source branch produces the velum and glottis extensions $E_{\text{source}}(t)$ of Eq. (1). Its guiding principle, stated in the code: *the source envelope has no timing of its own — it is indexed by the articulatory events already produced by $B(t)$; coarticulation is inherited.*

6.1 The three-level tag system

Phonetic knowledge is described by *tags*, not by raw VTL values, through three levels (`core/tag_system.py`):

Level A — phonetic tags: `voiced`, `nasal`, `continuant`, `strident`, `aspiration`, `place`, `manner`, `rounding`, height/depth, front/back, tense.

Level B — articulatory tags: `glottal_shape` (modal, voiceless, breathy, whisper, …), `velum` (oral, nasal, nasalized), `tongue_mode`.

Level C — control tags: glottal opening (0–1), subglottal pressure (200–2000 dPa), F0 (50–500 Hz), aspiration strength (0–30 dB), chink area (0–50).

A tag like `<glottal = voiceless>` is preferred to a hard-coded value like `VO = 0.42`: phonetic tags are compiled to speaker parameters at run time, which is what makes speaker recalibration (JD2/JD3) possible.

6.2 The amplitude envelope $E(t)$ and the source timetable

An *effort* (amplitude) envelope $E(t)$ is built from a token reduction of the phrase (vowels V, plosives C/c, fricatives R/r, nasals N, laterals L, silences O, final F, glide onsets y) with per-token minimum amplitudes (V 1.0, nasals and liquids 0.3, glides 0.5). The transitions between tokens use raised-cosine/Hanning primitives — e.g. a vowel-to-vowel junction stays at 1, a vowel-to-plosive falls as hann^3, a plosive-to-vowel rises as hann. Mapped onto the `syl` block schedule, plateaus and backgrounds get $E = 1$, pauses and terminals $E = 0$, clusters a reduced consonantal envelope, and decay/attack blocks the fading edges. The envelope is computed at 10 kHz and block-averaged down to 100 Hz.

On top of $E(t)$, a `TimedEnvelope` schedules *events* anchored to the articulatory labels (the vowel triplets (V_o, V, V_e), the consonant labels `C_p`, …) with raised-cosine ramps. The ramp constants (in ms): attack/release 30/30, voicing 40, velum 15, aspiration 8, F0 15, effort 20.

6.3 Voicing, VOT and nasality

Voicing $V(t)$ is built per syllable from the (V_o, V, V_e) triplets: voicing starts at the first voiced consonant of the syllable (the synthetic onset anchor V_o is purely articulatory — silent), stops 10 ms before a stop coda, ramps 30 ms in / 40 ms out; voiceless events then override inside obstruents. During cluster blocks, voiced consonants (stops, nasals, liquids, voiced fricatives) carry $V = 1$ — the pre-voicing bar of `/b, d, g/`.

Voice-onset time is set by place of articulation (Keating 1984; Lisker & Abramson 1964), in ms:

place	labio-dental	labial	dental	alveolar	post-alveolar	palatal	uvular	velar	glottal
VOT (ms)	25	20	30	40	40	35	45	50	60

Nasality is the velum opening VO of §4.7: 1.0 during nasal consonants, 0.5 on nasalized vowels, baseline −0.1, with the 20 ms carry-over and smoothing.

6.4 Glottal source, pressure and effort

The 11 glottis parameters interpolate between **presets extracted from the `JD3.speaker` file** (`modal` — the default, f0 102.216 Hz, pressure 7850 dPa; `stop`; `voiced-fricative`; `voiceless-fricative`; `whisper`; …), blended by $V(t)$ and smoothed with a second-order critically damped Butterworth filter (time constant 12 ms). The subglottal pressure follows the effort:

$$P_{\text{sub}} \;=\; P_{\text{min}} \;+\; \min(E, 1.5)^{1.2}\,(P_{\text{max}} - P_{\text{min}}), \qquad P_{\text{max}} = 8\,000\,\text{dPa}, \tag{9}$$

with a phonation floor of 3 000 dPa while voicing is on and a pre-voicing regime during voiced closures (pressure 1800 dPa, relative amplitude clipped to 0.10–0.18). The pulmonary-effort variant zeroes the effort on plosive occlusions (5 ms smoothing) and takes the maximum of the effort, 60 % of the pre-voicing level and 50 % of nasality before smoothing (10 ms).

A per-vowel effort gain compensates the intrinsic intensity deficit of high vowels: `i, u, e` ×1.5, `y` ×1.3, `2` ×1.2 on their plateaus (≈ +3 dB).

6.5 F0: declination and micro-prosody

The default F0 is the JD3 speaker default 102.216 Hz. Each phrase receives a French-style **declination** (Jun & Fougeron 2002): +3 % at onset, −15 % at the end, linear in between; phrases are detected as voiced spans separated by more than 200 ms of silence. Superimposed on it, a **micro-prosody** couples F0 to the effort envelope:

$$F0(t) \;=\; F0_{\text{base}} \;+\; k_E\,\big(E(t) - \overline{E}\big), \qquad k_E = 10\,\text{Hz}, \tag{10}$$

clamped to $[40, 600]$ Hz, with the last voiced value held through unvoiced spans.

7 The multi-speaker registry and the language pack

Since v1.1.0 the package carries five speakers and six languages. Both selectors follow the same design: a *registry* on disk is the single source of truth, and one reversible command swaps the installed state; `restore` always returns to a bit-exact previous state.

7.1 The speaker registry

The directory `vtl_synth/data/speakers/` holds `registry.json` (one entry per speaker: file names, `f0_default`, description, SHA-256 of both files), the `ACTIVE_SPEAKER` marker, the five `.speaker` files and their `_constants.py` sources. The installed `core/constants.py` is always a bit-exact copy of a registry source, so its hash identifies the speaker.

name	origin	sex	f0_default (Hz)	vowel targets
jd3	VTL 2.4 reference	male	102.216	follows the language pack
s1	DVTD subject 1 (MRI)	male	114.908	native (v3 constants)
s2	DVTD subject 2 (MRI)	female	181.008	native (v3 constants)
m01	official VTL 2.4 (Option A)	male	120.000	native (v3 constants)
w02	official VTL 2.4 (Option A)	female	232.458	native (v3 constants)

A switch coordinates four layers: the installed `constants.py` (geometry, glottis, `f0_default`), the `.speaker` file read by the orthogonal branch at each call, the shared speaker resource of the `vocaltractlab-cython` wheel (effective in the next process), and the per-speaker regression baselines (`regression_baselines_{s1,s2,m01,w02}.json`, regenerated by `scripts/regen_baselines.py`).

```
python install_speaker.py list            # speakers + active one
python install_speaker.py install s2      # 7-step swap + smoke test
python install_speaker.py restore         # back to jd3 (bit-exact)
python install_speaker.py status          # hashes, layer coherence
```

Every `install` runs a smoke test in a fresh subprocess and rolls back automatically on any error; every synthesis announces the four layers (constants, ortho, audio wheel, video) and warns on any disagreement. Provenance of the five speakers, the two adaptation chains and the defect catalogue are documented in `docs/speakers.pdf`.

7.2 The language pack

`setup_lang.py` manages a delimited LANG SECTION (`VOWEL_TARGETS`, `VOWEL_EFFORT_GAIN`, `ACTIVE_LANG`) inside the installed `constants.py`, plus per-language g2p profiles, lexicons (Wikipron) and prosodic timings:

```
python setup_lang.py install --lang fr    # install + activate
python setup_lang.py setlang en|fr|es|de|it|pt
python setup_lang.py calibrate fr --in-situ   # JD3 only (guard)
python setup_lang.py verify               # in-situ LPC check
python setup_lang.py restore --purge      # full uninstall
```

Two invariants matter for day-to-day use. **Only JD3 follows the pack**: the per-language vowel targets are calibrated in situ on JD3, and the language-management commands carry a guard that (re)installs jd3 before touching the LANG SECTION; the other speakers are used with their native targets. **Per-call override**: the CLI flag `-l` / `--lang` selects the language of a single call without switching the installation. Installation, calibration, verification scores and troubleshooting are documented in `docs/language_pack.pdf`; the expressive-prosody option (`--expressive`) and its interaction with the registry in `docs/expressive_prosody.pdf`.

8 Architecture and data flow

The package is organized into five subpackages:

`vtl_synth.core` the synthesis engine: notation/parsing, phoneme tables, the polar model, the trajectory engines, the envelope/source modules, the `Pipeline` orchestrator, and the speaker registry resolver (`speaker_registry.py`).

`vtl_synth.cli` the `vtl-synth` entry point with the subcommands `run`, `text-to-wav`, `tract-to-mp4`, `polar-build`, `polar-video`, `inspect`, `plot-tract`.

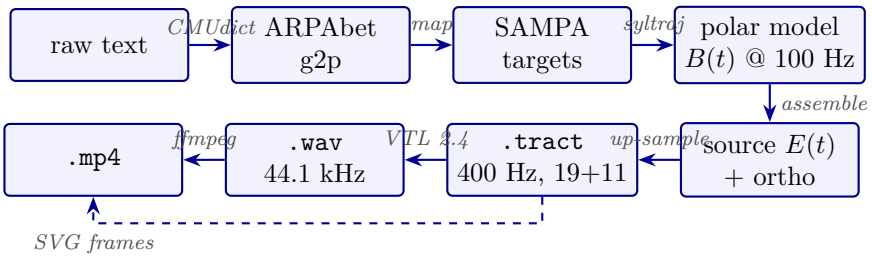
`vtl_synth.vtl` the VocalTractLab API facade (`api.py`) over the `vocaltractlab-cython` wheel.

`vtl_synth.video` the sagittal renderer (`renderer.py`: SVG frames → PNG), the MP4 encoder (`encoder.py`: ffmpeg H.264/AAC), the parameter figure tool (`tract_figure.py`), and the polar-figure module (`polar_video.py`: two-branch reconstruction, dual-panel composition).

`vtl_synth.utils` the grapheme-to-phoneme modules (English `g2p.py` on CMUdict; the five language-pack profiles), the language-profile layer (`setlang.py`, `lexicon_loader.py`) and the prosody add-ons (`chunking.py`, `prosody_f0.py`).

`vtl_synth.data.speakers` the speaker registry: `registry.json`, the `ACTIVE_SPEAKER` marker, the five `.speaker` files and their `_constants.py` sources (§7).

8.1 Data flow



Each stage of the chain:

1. **g2p**: English text → connected engine SAMPA tokens (§11); or direct SAMPA with `--phonetic`.
2. **Parse & normalize**: Unicode/IPA normalization, longest-match tokenization, phrase splitting on `|`, syllable structure.
3. $B(t)$: the `syl` engine builds the 100 Hz polar trajectory block by block (§5), projected through Eq. (3) into the 15 COVTL parameters.
4. **Source**: the envelope $E(t)$, voicing $V(t)$, velum VO/VS, glottis trajectory and F0 (§6), all indexed by the block schedule; the orthogonal branch adds TS3/VS (§5.1).
5. **Assemble**: the 15 COVTL parameters are scattered into the 19-parameter VTL vector (VS at index 6, VO at 7; TS3 additive; TRX/TRY left at 0 — VTL drives them), the glottis block is appended, and everything is linearly up-sampled 100 → 400 Hz into the `.tract` file.
6. **VTL synthesis**: each tract state → area function (40 tube sections, JD3 anatomy) → aerodynamic-acoustic simulation → 110 audio samples per state at 44.1 kHz.

7. **Video**: every video frame is exported as an SVG of the sagittal cross-section by VTL itself, rasterized to PNG, and encoded with the bundled ffmpeg.

Three sampling rates

- **Model rate**: 100 Hz (gesture step 10 ms) — the polar trajectory and the envelopes.
- **Tract rate**: 400 Hz (`.tract`) — 19 tract + 11 glottis parameters per frame, 2.5 ms hop.
- **Audio rate**: 44 100 Hz — 110 audio samples per tract state (`.wav`, PCM 16-bit mono).

9 CLI reference

The `vtl-synth` command exposes seven subcommands: `run`, `text-to-wav`, `tract-to-mp4`, `polar-build`, `polar-video`, `inspect` and `plot-tract`. The engine options (COVTL model parameters) are shared by `run` and `text-to-wav`:

Engine flag	Description
`--phonetic`	input is already engine SAMPA (skip g2p)
`-l, --lang CODE`	language for this call only (default: the active language of the pack, §7)
`--tcons MS`	consonant (sub-arc) duration, default: active language profile (en: **100 ms**)
`--tvoy MS`	vowel gesture duration, default: active language profile (en: **80 ms**)
`--tpause MS`	short inter-word pause (space), default: active language profile (en: **200 ms**)
`--tpause-long MS`	long phrase pause (|), default: active language profile (en: **320 ms**)
`-f0, --f0 HZ`	fundamental frequency, default: `f0_default` of the active speaker (102.216 Hz for JD3)
`-s, --speaker`	speaker name, default `JD3`
`--expressive` / `--no-expressive`	expressive prosody (breathing + sculpted F0) / monotone declination (default); see `docs/expressive_prosody.pdf`
`--no-ortho`	disable the orthogonal branch (TS3/VS)
`--engine {syl,legacy}`	trajectory engine, default `syl`

9.1 run — all-in-one synthesis

```
vtl-synth run "this is easy for us" [options]
```

Runs the full pipeline: text → `.tract` → `.wav` → `.mp4` (+ the SAMPA transcript `.txt`).

Flag	Description
`-o, --output-dir`	output directory (default: `./out`)
`--label`	filename stem (default: derived from the text)
`--no-video`	skip MP4 rendering
`--polar`	also write the `.polar` file and render the dual-panel video (tract | polar figure); see `docs/polar_visualization.pdf`
`--fps`	video frame rate (default: 25)
`--scale`	pixels per SVG unit (default: 2 → 960×840)

```
# Direct phonetic input (no g2p): syllables '.', short pause ' ', long '|'
vtl-synth run --phonetic "ba da ga | iowa a g" -o ./out

# Slower, more coarticulated speech
vtl-synth run "this is easy for us" -o ./out --tcons 140 --tvoy 110

# Audio only
vtl-synth run "hello world" -o ./out --no-video

# French, expressive prosody (breathing + sculpted F0)
vtl-synth run "bonjour le monde" -l fr --expressive -o ./out --no-video

# Dual-panel polar video (en/jd3)
vtl-synth run --polar "this is easy for us" -l en -o ./out --label polar_demo_en
```

9.2 text-to-wav — audio-only synthesis

```
vtl-synth text-to-wav "hello world" -o ./out
```

Same engine path as `run`, stops after the `.wav`.

9.3 tract-to-mp4 — re-render a video

```
vtl-synth tract-to-mp4 <tract> [<wav>] [-o output.mp4] [--fps 25] [--scale 2]
```

Re-renders the sagittal video from an existing `.tract`; when the `.wav` is omitted it is re-synthesized from the tract. Useful to change resolution or frame rate without re-running the model.

9.4 polar-build / polar-video — the polar figure

```
# .polar only, from a phrase in the internal notation
vtl-synth polar-build "ba.da.ga" -o out/ba.polar --tcons 160

# Dual-panel video from existing files
vtl-synth polar-video out/x.tract --polar out/x.polar out/x.wav -o out/x.mp4
```

`polar-build` reconstructs the two 100 Hz polar trajectories (vocalic branch / consonantal branch) of a phrase and writes the JSON `.polar` side-car; `polar-video` composes the dual-panel MP4 (sagittal view | polar figure, 25 Hz) from a `.tract`, a `.polar` and a `.wav`. The reconstruction, the file format and the rendering are documented in `docs/polar_visualization.pdf`.

9.5 inspect — tract inspector

```
vtl-synth inspect ./out/this_is_easy_for_us.tract --states 2 --ranges
```

Prints the shape, rate and duration of the tract sequence, the first/last N states as per-parameter tables, and with `--ranges` the min/max/mean of each of the 19 tract parameters.

9.6 plot-tract — graphical inspector

```
vtl-synth plot-tract ./out/this_is_easy_for_us.tract --which all -o
    ./out/tract.png
vtl-synth plot-tract ./out/this_is_easy_for_us.tract --params JX JA TTX TTY
    --ranges
```

Draws the 400 Hz trajectory of every tract parameter into a PNG figure (shared time axis, one subplot per parameter); `--which all` adds the glottis f0 and pressure, `--params` restricts the tract block, `--ranges` prints the `inspect` table. Requires the optional `plot` extra (`pip install ".[plot]"`).

10 Python API reference

10.1 Pipeline — high-level orchestrator

```
from vtl_synth import Pipeline

pipe = Pipeline(
    f0_hz=102.216,         # JD3 default; per-phrase declination applied
    speaker="JD3",
    engine="syl",          # 'syl' (recommended) or 'legacy'
    use_orthogonal=True,   # TS3/VS orthogonal branch
    t_cons_ms=100.0,       # consonant sub-arc duration
    t_voy_ms=80.0,         # vowel gesture duration
    pause_short_ms=200.0, # inter-word pause
    pause_long_ms=320.0,   # end-of-sentence pause
)

result = pipe.run("this is easy for us", output_dir="./out")
print(result.sampa)               # 'Dis.iz.i.zi.fOR.@s'
print(result.tract_path, result.n_frames, result.duration_s)

# phonetic input, audio only
result = pipe.text_to_wav("ba da ga | iowa a g",
                          output_dir="./out/phon", use_g2p=False)
```

The `PipelineResult` dataclass exposes `sampa`, `phrases`, the artifact paths (`tract_path`, `wav_path`, `mp4_path`), `n_frames` (at 400 Hz), `duration_s` and `warnings`.

10.2 Video re-rendering

```
info = Pipeline.tract_to_mp4(
    tract_path="./out/this_is_easy_for_us.tract",
    wav_path="./out/this_is_easy_for_us.wav",
    out_path="./out/regenerated.mp4",
    fps=25, scale=2,
)
```

This does *not* re-run the model: it parses the tract sequence, renders one SVG frame per video frame and encodes with ffmpeg.

10.3 Engine and VTL APIs

The full engine API (63 public symbols) is available from `vtl_synth.core`: the orchestrators (`Pipeline`, `build_phrase_tract`, `text_to_tract`), the polar model (`compute_P`, `arc_B`, `polar_arc`, `project_covtl`, `superimpose`), the trajectory engines (`build_phrase_pval`, `build_global_pval`), the envelope/source modules (`TimedEnvelope`, `FeatureTimer`, `build_glottis_trajectory`), the constants (`VOWEL_TARGETS`, `CONSONANT_TARGETS`, `CO_VTL`, `CO`) and the assembly layer (`AssembleTract`, `write_tract_file`).

```
from vtl_synth.vtl.api import (synthesize_to_wav, render_tract_svg,
                               get_vtl_info, get_vowel_shape,
                               get_transfer_function)

info = get_vtl_info()
# {'version': ..., 'speaker': 'JD3', 'sr_audio': 44100,
#  'sr_internal': 400.0, 'n_tract_params': 19,
#  'n_glottis_params': 11, 'n_samples_per_state': 110,
#  'n_tube_sections': 40, ...}

synthesize_to_wav("./out/this_is_easy_for_us.tract",
                  "./out/resynthesized.wav")
```

11 Input notation and grapheme-to-phoneme conversion

11.1 Engine SAMPA inventory

- **Vowels**: `a e i o u y E O @ 2 6 9` (12 targets, Table 1);
- **Consonants**: `p b t d k g f v s z S Z T D tS dZ m n J l L R j w h ɥ C X` (canonical voiced forms of Table 2 plus their voiceless partners; `R` is the only rhotic — a spelled `r` in the input is normalized to `R` by `notation.py`);
- **Separators**: `.` syllable break (coarticulated), space short pause, `|` long pause; `~` marks nasality.

11.2 English text rules

With English text input (the default), the bundled g2p applies:

1. CMUdict lookup of each word (first pronunciation), stress digits stripped;
2. ARPAbet → engine SAMPA mapping (Table 5);
3. syllabification by onset maximization: the vowel closes the nucleus, a single intervocalic consonant attaches to the next onset, longer clusters split before the last consonant, word-final consonants form the coda;
4. the words of a phrase are joined with `.` into connected tokens — the engine coarticulates across word boundaries (§4.6).

Punctuation: sentence-ending punctuation (`.!?;:`) becomes the long phrase pause `|`; a comma or dash becomes the short inter-word pause. Unknown words that are already valid engine SAMPA (`is_sampa_token`) are passed through verbatim; other unknown words are kept with a warning.

ARPAbet	SAMPA	Note
P B T D K G	p b t d k g	direct (voiceless resolves to voiced target)
CH, JH	tS, dZ	affricates
F V S Z SH ZH	f v s z S Z	direct
TH, DH	T, D	dental fricatives — engine targets exist
HH	h	direct
M N NG	m n J	nasals; `NG`→`J` is a documented compromise: the English velar nasal [ŋ] is mapped to the derived dorsal node (neither `N` nor `J` carries a closure target, Table 2)
L R W Y	l R w j	R is the uvular of the engine inventory
AA AE	a	AE merges to `a`
AH	@	schwa
AO	O	
EH EY	E, e	
IH IY	i, i	quantity not distinguished
OW	o	
UH UW	u, u	quantity not distinguished
AW AY OY ER	a w, a j, O j, @ R	diphthongs/rhotic → key sequences

Table 5: ARPAbet → engine SAMPA mapping (`ARPA_TO_SAMPA`, `core/phonemes.py`).

12 File formats reference

12.1 `.tract` — tract sequence at 400 Hz

Plain-text file: comment lines starting with `#`, a fixed header (sample rate, parameter counts and names, frame count), then one line per frame with the 19 tract + 11 glottis values (`%.6f`):

```
# VTL tract sequence file
# version: 1
# sample_rate: 400
# n_tract_params: 19
# n_glottis_params: 11
# n_frames: 952
# tract_param_names: HX HY JX JA LP LD VS VO TCX TCY TTX TTY TBX TBY
#                    TRX TRY TS1 TS2 TS3
# glottis_param_names: f0 pressure x_bottom x_top chink_area lag rel_amp
#                    double_pulsing pulse_skewness f0_flutter
#                    aspiration_strength
# ---
<frame 0: 19 tract values, then 11 glottis values>
...
```

The 19 tract parameters: hyoid `HX HY`, jaw `JX JA`, lips `LP LD`, velopharyngeal port `VS VO` (extensions), tongue center `TCX TCY`, tongue tip `TTX TTY`, tongue base `TBX TBY`, tongue root `TRX TRY` (auto-driven, kept at 0), tongue shape `TS1 TS2 TS3`. The 11 glottis parameters: `f0` (Hz), `pressure` (dPa), `x_bottom`, `x_top`, `chink_area` (posterior opening/breathiness), `lag`, `rel_amp`, `double_pulsing`, `pulse_skewness`, `f0_flutter` (%), `aspiration_strength` (dB).

The reader is `read_tract` (`vtl_synth.video.renderer`); it parses `# sample_rate:` and expects at least 19 columns.

12.2 `.wav`, `.mp4`, `.txt`

- `.wav`: PCM 16-bit mono at 44 100 Hz, peak-normalized to 0.9 full scale by the pipeline.

- `.mp4`: H.264 (`yuv420p`) + AAC; default 960×840 at 25 fps (`--scale 2`). The upper panel is the sagittal cross-section exported by VTL as SVG; the audio track is muxed by ffmpeg (`-shortest`).
- `.txt`: the engine SAMPA transcript actually synthesized (e.g. `Dis.iz.i.zi.fOR.@s`) — the audit trail of the g2p.
- `.polar` (optional, `--polar`): JSON side-car carrying the two 100 Hz polar trajectories (vocalic / consonantal, `null` while a branch is inactive) and the phoneme timing; consumed by the dual-panel video (`docs/polar_visualization.pdf`).

13 Worked example: "this is easy for us"

This section walks through the real output of the demo phrase in the default configuration (speaker jd3, language en; all numbers measured on the artifacts of a default run).

13.1 g2p output

```
word   ARPAbet (CMUdict)   engine token
this   DH IH1 S            Dis
is     IH1 Z               iz
easy   IY1 Z IY0           i.zi
for    F AO1 R             fOR
us     AH1 S               @s
```

The words of the phrase are connected with `'.'`, giving the single engine token `Dis.iz.i.zi.fOR.@s`. Note the resyllabifications (*is easy* → `iz.i.`, *easy* split `i.zi`): one coarticulated phrase, no pause at the orthographic word boundaries. The full SAMPA transcript written to `.txt`: `Dis.iz.i.zi.fOR.@s`.

13.2 Tract sequence

```
shape     : (952, 30)  (11 glottis cols)
rate      : 400 Hz
duration  : 2.380 s
```

952 frames at 2.5 ms hop. The first and last frames are the resting state — the projection of the phrase-terminal anchors $[0, 0]$ is the center line of Table 3, e.g. `HX = 0.722`, `JA = −3.371`, `TCX = 1.091`.

13.3 Audio and video

```
wav : PCM 16-bit, 1 channel, 44100 Hz, 104720 frames, 2.3746 s
      peak 90.00 % FS (pipeline normalization), RMS 7.6 % FS
mp4 : h264 (High), yuv420p, 960x840, 25 fps, 2.40 s
      + aac 44100 Hz mono; 239,001 bytes
```

The video and audio durations agree to within one video frame (2.40 vs 2.37 s), confirming the sync (each frame decimates the 400 Hz tract sequence by $400/25 = 16$ frames).

14 Differences with the vtl-pipeline reference repository

The vtl-pipeline reference repository chains VTL-native stages (text → `.seg` → `.ges` → tract → wav/mp4) through a ctypes binding to `libVocalTractLabApi`. covtl-pipeline shares its packaging and CLI layout but synthesizes the tract sequence **directly** with the Berthommier COVTL model:

	vtl-pipeline (reference)	**covtl-pipeline**
Tract generation	VTL segment → gestural score	COVTL polar model, no `.seg`/`.ges`
Coarticulation	VTL gesture blending	selector superposition (§4)
Targets	VTL shape library	(ρ, θ) plane (§3)
VTL backend	ctypes `.so` + Krug wheel	`vocaltractlab-cython` wheel
Input	English text	English text or SAMPA (`--phonetic`)

Consequences: no `.seg`/`.ges` files exist in the output (`inspect` rejects them with an explanatory message), no manual DLL/library management is needed, and the articulatory trajectory is fully transparent — every 100 Hz model sample derives from the documented equations of §3–§4.

15 Limitations and known issues

- **English phoneme approximations.** The engine works on a French-style SAMPA inventory: `R` is uvular (English rhotics are approximated), `AE/AH` merge to `a/@`, quantity contrasts (`IH/IY`, `UH/UW`) are not distinguished, and `ER` becomes the sequence `@ R`. Dental fricatives `TH/DH` do have engine targets (`T/D` at 271°).
- **Fixed gesture durations.** One `--tcons`/`--tvoy` pair applies to the whole utterance; there is no per-phoneme duration model, no stress-weighted durations and no word-final lengthening beyond the phrase-final hold.
- **Derived places.** `N` and `J` have empty selectors (derived velar/dorsal places); `h` is a near-neutral target driven by the source.
- **Legacy engine deprecated.** `--engine legacy` emits a `DeprecationWarning` and is kept only for non-regression.
- **Video renderer.** The SVG → PNG rasterizer is a lightweight polyline renderer built on Pillow (no cairo dependency); complex SVG features beyond the VTL sagittal output are not supported.
- **Python 3.9 compatibility mode.** With `vocaltractlab-cython==0.0.13`, the four analysis helpers added in 0.0.16+ (`get_cross_sections`, `get_centerline`, `get_outlines`, `get_active_speaker`) raise a clear error; synthesis and video are unaffected.

16 Developer guide

16.1 Package layout

```
covtl-pipeline/
|-- pyproject.toml              PEP 621 metadata (Python >= 3.9), GPL-3.0-or-later
|-- LICENSE                     GPL-3.0 full text
|-- THIRD_PARTY_NOTICES.md      upstream author credits
|-- README.md, INSTALL.md, CHANGELOG.md
|-- launch.bat                  Windows quick-launch script
|-- install_speaker.py          speaker selector (list/install/restore/status)
|-- setup_lang.py               language selector (install/setlang/...)
|-- lang_pack/                  language pack source tree, lexicons, manual
|-- regression_baselines.json           E2E baselines, speaker jd3
|-- regression_baselines_{s1,s2,m01,w02}.json  per-speaker baselines
|-- examples/                   demo.py, demo_prosody.py
|-- scripts/                    regen_baselines.py, calibrate_fine_v2.py,
|                               f0_report.py, check_vowel_area.py
|-- tests/                      lightened quick suite (see below)
|-- docs/                       the five manuals (LaTeX + PDF)
`-- vtl_synth/
    |-- __init__.py             public API exports (Pipeline, ...)
    |-- core/                   engine modules (model, trajectories,
    |                           source, speaker_registry)
    |-- cli/main.py             vtl-synth entry point (7 subcommands)
    |-- video/                  renderer, encoder, tract_figure, polar_video
    |-- vtl/api.py              VocalTractLab API facade
    |-- utils/                  g2p, language profiles, prosody add-ons
    `-- data/
        |-- vtl_binaries/JD3.speaker     package speaker data
        `-- speakers/           registry.json, ACTIVE_SPEAKER,
                                *.speaker, *_constants.py (5 speakers)
```

16.2 Running the tests

```
pip install -e ".[dev]"
pytest -v                                       # quick suite
pytest tests/test_regression_baselines.py       # per-speaker E2E baselines
pytest tests/test_speaker_registry.py           # registry integrity
```

The repository ships a lightened quick suite: end-to-end non-regression against the per-speaker baseline files (`regression_baselines.json` for jd3 and one baseline file per other speaker), speaker-registry integrity, and fast CLI/g2p unit tests. The full development battery (polar model, syltraj structure, notation/parsing, glottal classification, expressive prosody, multi-language g2p, tract figure; 112 tests in 10 modules on the reference development installation) runs from the development tree with `pytest -m "not slow"`.

16.3 Extending the model

- **New phoneme target**: add the (ρ, θ) entry (and selector) to `CONSONANT_TARGETS` or to `VOWEL_TARGETS` in `core/constants.py` — the tokenizers pick it up from `ALL_PHONEME_KEYS` automatically.
- **New pause/gesture timing**: the CLI flags map to `Pipeline` arguments; the block schedule constants of the `syl` engine are documented in `syltraj.py`.
- **Coding conventions**: see `docs/CONVENTIONS.md` (header format, module docstrings, validation style).

- **SPDX headers** (`SPDX-FileCopyrightText`, `SPDX-License-Identifier`) are present in every `.py` file (GPL-3.0-or-later) and must be preserved in contributions; `THIRD_PARTY_NOTICES.md` must be updated whenever a new upstream dependency is added.

17 License, acknowledgments and references

covtl-pipeline is distributed under the GNU General Public License v3 or later (GPL-3.0-or-later), Copyright (C) 2026 Frédéric Berthommier. The GPL is required because the package depends on GPL-3.0 upstream components:

- **VocalTractLab** — Peter Birkholz, TU Dresden. https://www.vocaltractlab.de
- **vocaltractlab-cython** — Paul Krug. https://github.com/paul-krug/VocalTractLab-Python
- concepts from **create_vtl_corpus** — Konstantin Sering, Universität Tübingen. https://github.com/quantling/create_vtl_corpus
- **VTL 2.4 speaker files** (JD3 template; M01, W02 redistributed in patched/reformatted form with attribution) — official `VTL2.4.zip`, https://www.vocaltractlab.de.
- **Dresden Vocal Tract Dataset** (speakers s1/s2) — Birkholz et al., *Scientific Data* 7:255, 2020; CC0 1.0.
- **Wikipron lexicons** (fr, es, de, it, pt) — CC BY-SA 3.0 (attribution notices in `lang_pack/LICENSE.md`).

The `JD3.speaker` file is redistributed as package data (`vtl_synth/data/vtl_binaries/`); `cmudict` is BSD-2. When redistributing, keep the full `LICENSE` and `THIRD_PARTY_NOTICES.md` intact and publish modifications under GPL-3.0-or-later.

References

If you use covtl-pipeline in academic work, please cite the publications of the Berthommier COVTL model:

1. F. Berthommier, "A mathematical model of the vowel space", arXiv:2111.00868 [eess.AS], 2021. DOI: 10.48550/arXiv.2111.00868.
2. F. Berthommier, "Why can big.bi be changed to bi.gbi? A mathematical model of syllabification and articulatory synthesis", arXiv:2307.02299 [eess.AS], 2023. DOI: 10.48550/arXiv.2307.02299.
3. F. Berthommier, "Synthèse de syllabes avec un modèle de Maeda piloté par une représentation complexe", in *Actes des 35èmes Journées d'Études sur la Parole (JEP)*, pp. 541–550, ATALA and AFPC, 2024. https://aclanthology.org/2024.jeptalnrecital-jep.55/
4. F. Berthommier, "Articulatory modeling of the S-shaped F2 trajectories observed in Öhman's spectrographic analysis of VCV syllables", *Interspeech 2025*, Rotterdam, Netherlands. HAL: hal-05233039. https://hal.science/hal-05233039v1

Please also cite VocalTractLab itself (see `THIRD_PARTY_NOTICES.md` for the up-to-date list).